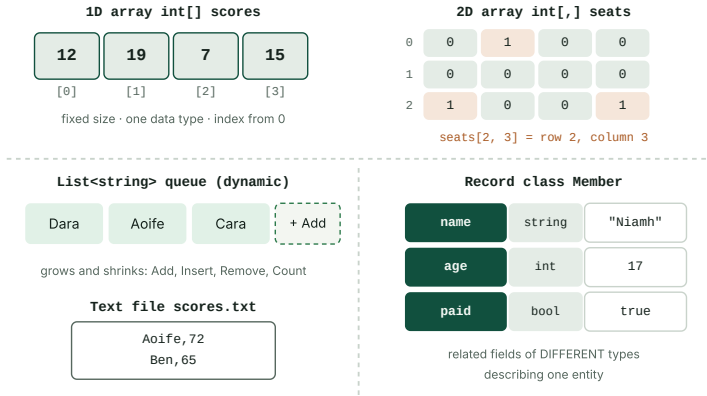
Data structures: arrays, lists, records and files

AP22 FACT FILE

Specification: students should be able to

- Describe the purpose of data structures and distinguish between static and dynamic structures
- Use one-dimensional and two-dimensional arrays, including traversing them with loops
- Use lists as dynamic data structures and justify the choice of a list or an array
- Describe and use records to group related fields of different data types
- Write to, append to and read from text files, splitting delimited lines into fields
- Explain why files are needed for persistent storage and how file errors can be avoided

Key words: data structure, array, index, element, two-dimensional array, static, dynamic, list, record, field, struct, traverse, text file, delimiter, CSV, persistent storage, append



Four data structures: a one-dimensional array with indexes, a two-dimensional array addressed by row and column, a growing list, and a record with fields of different types

Overview: storing collections of data

Most real programs handle collections of data rather than single values: the marks of a class, the seats in a theatre, the members of a club. A data structure groups these items under one identifier so that a loop can process them all with a few lines of code. This lesson covers four structures. An array stores a fixed number of elements of one data type, accessed by an index that starts at 0; a two-dimensional array arranges elements in rows and columns. A list is also a collection of items of one type, but it is dynamic, so items can be added and removed while the program runs. A record groups related fields of different types, such as a member's name, age and whether fees are paid, so that one item can be described completely. Finally, a text file stores data on secondary storage so that it survives after the program closes. Choosing well matters: the right structure makes code shorter, easier to read and less likely to contain errors, and exam questions often ask you to justify a choice for a given scenario.

One-dimensional arrays

An array is a static data structure: its size is fixed when it is created and every element has the same data type. In C# the declaration int[] goals = new int[5]; reserves space for five integers, each set to the default value 0, while an initialiser list such as { 4, 0, 12 } sets both the size and the values. Each element is reached directly through its index, so goals[0] is the first element and goals[goals.Length - 1] is the last. Direct access by index makes arrays fast, because the program does not need to look through earlier elements to reach a later one. A for loop with the counter running from 0 while it is less than Length is the standard way to traverse an array, for example to total the values, find the largest, count items that meet a condition or carry out a linear search. Trying to use an index outside 0 to Length - 1 causes an IndexOutOfRangeException at run time, which is one of the most common errors when working with arrays.

Two-dimensional arrays

A two-dimensional array stores data that naturally forms a grid or table, such as tickets sold for several shows over several nights, a seating plan, or the scores of each pupil in each test. Every element still has the same data type, and it is identified by two indexes, conventionally written [row, column]. In C# the declaration int[,] sales = new int[3, 4]; creates three rows of four columns, twelve elements in all. GetLength(0) returns the number of rows and GetLength(1) the number of columns, while Length returns the total number of elements. Processing a 2D array normally needs nested loops: the outer loop selects a row and the inner loop moves across each column of that row. To total each row, a running total is set to zero at the start of each pass of the outer loop and output or stored when the inner loop finishes; to total each column the loops are simply swapped. Keeping the order of the indexes consistent throughout a program is essential, because swapping them gives wrong results or an index out of range error.

Lists and choosing between a list and an array

A list is a dynamic data structure: it starts empty, or with some items, and grows or shrinks as the program runs. In C# a List<string> holds strings and a List<Member> holds Member records. Built-in methods make common tasks simple: Add puts an item at the end, Insert places it at a given index, Remove deletes the first matching item, RemoveAt deletes by position, Contains and IndexOf search, and Sort puts items in order. Count gives the current number of items. An array is the better choice when the number of items is known in advance and will not change, such as twelve monthly totals or a fixed board for a game, because it is simple and its memory is reserved once. A list is the better choice when the amount of data is unknown or varies, such as bookings taken during the day, because the program does not have to guess a maximum size, waste memory on unused elements or copy data into a bigger array. When justifying a choice, always link the property of the structure to the scenario.

Records

A record is a data structure made up of a fixed number of named fields that together describe one entity, such as a player, a pupil or a product. Unlike an array, the fields of a record may be of different data types: a player record might contain a string name, an integer squad number, a real height and a Boolean injured flag. In C# a record type can be defined with a struct or with a class that has public fields, and each field is accessed with dot notation, as in p.squadNumber. Grouping the fields keeps all the data about one item together, so it can be passed to a subroutine, stored or sorted as a single unit. The alternative, several parallel arrays such as names[] and ages[] that share an index, works but is error-prone because the arrays can easily fall out of step. An array or list of records behaves like a table in a database, with each record forming a row and each field a column, and it maps directly onto a line of a CSV file, which makes saving and loading data straightforward.

Text files and file handling

Data stored in variables, arrays and lists is held in RAM and is lost when the program ends or the computer is switched off. Text files on secondary storage provide persistent storage so that data can be reloaded the next time the program runs. A program opens a file in a particular mode. Opening for writing creates the file, or overwrites it completely if it already exists; opening for appending adds new lines to the end without disturbing existing data; opening for reading lets the program process the existing lines. A common layout is one record per line, with fields separated by a delimiter such as a comma, which is known as a CSV file. When a line is read, Split separates it into an array of strings, and numeric or Boolean fields must then be converted with methods such as int.Parse before they are used in calculations or conditions. Files must be closed after use so that buffered data is actually written to the disk and the file is released; a using block does this automatically. Robust programs also check that a file exists before reading it, or handle the exception that occurs if it does not.

T Key terms from the mark schemes

Data structure
A way of organising and storing a collection of related data items under one identifier so that they can be processed efficiently.

Array
A static data structure that holds a fixed number of elements of the same data type under one identifier, each accessed by an index.
`Credited: a set of data items of the same type grouped under one name`

Index
The position number used to access an element of an array or list; in C# the first index is 0.
Credited: subscript

Two-dimensional array
An array whose elements are arranged in rows and columns and are accessed using two indexes, [row, column].
Credited: 2D array; table/matrix

Static data structure
A data structure whose size is fixed when it is created and cannot change while the program runs, such as an array.

Dynamic data structure
A data structure whose size can grow or shrink while the program is running, such as a list.

List
A dynamic data structure whose size can change at run time as items are added or removed, with methods such as Add and Remove.

Record
A data structure that groups a fixed set of related fields, which may be of different data types, to describe one item.

Field
One named item of data within a record, such as a member's name or age.
Credited: attribute

Text file
A file of characters, usually organised in lines, stored on secondary storage so that data persists between program runs.

Delimiter
A character, such as a comma, used to separate the fields of a record stored on one line of a text file.
Credited: separator

Persistent storage
Storage that keeps data after the program ends or power is removed, such as a file on secondary storage.
Credited: non-volatile storage

C C# example

Declaring a one-dimensional array

```
int[] goals = new int[5];                 // five ints, all 0
string[] counties = { "Antrim", "Armagh", "Down",
                      "Fermanagh", "Londonderry", "Tyrone" };
double[] prices = new double[] { 2.50, 3.75, 1.20 };

goals[0] = 3;
goals[4] = goals[0] + 1;
Console.WriteLine(counties[3]);               // Fermanagh
Console.WriteLine(counties.Length);           // 6
Console.WriteLine(goals[4]);                  // 4
Console.WriteLine(prices[prices.Length - 1]); // 1.2
```

Exam focus

Definitions are usually worth one or two marks: for an array, mention a fixed number of elements, one data type, one identifier and access by index. For a record, stress related fields of different data types. Describe and explain questions about choosing a structure reward a property linked to the scenario, for example 'a list, because the number of bookings changes during the day' or 'a 2D array, because the data forms a grid of rows and seats'. Static versus dynamic questions need both sides: size fixed when created versus size can change while the program runs, ideally with an example of each. Code questions earn marks for a correct declaration, a loop with correct bounds, correct indexing and correct output, so write i < Length and start from 0. In file questions, credit is given for opening the file in the right mode, reading or writing a line at a time inside a loop, splitting on the delimiter, converting data types and closing the file. Common mistakes include starting indexes at 1, using <= Length, swapping row and column indexes, and forgetting that data read from a file is text.

? Check yourself

1. Define the term array. [2]
2. Explain the difference between a static and a dynamic data structure, giving a C# example of each. [4]
3. A youth theatre in Derry~Londonderry has 8 rows of 12 seats. Write a C# declaration of a suitable data structure to record whether each seat is booked, and a statement that marks the third seat in the first row as booked. [3]

Answers: 1. A data structure that stores a fixed number of elements/values (1) / of the same data type under one identifier, each accessed by an index (1). 2. Static: size is fixed when the structure is created and cannot change while the program runs (1) / e.g. int[] marks = new int[30]; (1) / Dynamic: size can grow or shrink at run time as items are added or removed (1) / e.g. List<string> names = new List<string>(); (1). 3. bool[,] seats = new bool[8, 12]; (accept int[,] or char[,] with explanation) - correct type and 2D syntax (1) / correct dimensions 8 and 12 (1) / seats[0, 2] = true; (1).